

Secure DevOps Pipeline

Building a CI/CD pipeline where security failures are physically unable to ship

Six automated gates · one locked door · verifiable proof

BCSSL Cybersecurity Lab Series — Project 15

The problem we are solving

why security usually happens far too late

- **Old way:** write code, build it, ship it. If it had a flaw, you found out from a customer.
- **The cost curve:** a bug caught at your keyboard costs minutes. The same bug in production costs an incident call, a patch, and trust.
- **Why nobody fixes it:** security reviews happen at the end, when the release is already built and everyone is tired.
- **The fix:** move the checks to the cheapest possible moment — the moment you push.

THE ONE IDEA

A gate is not a report someone reads later. It is a locked door wired to the checks. Nobody has to remember anything.

What we built, with one picture

the conveyor belt and the iron gate

The pipeline

six independent gates, then a door that will not open unless all six pass



A failure anywhere to the left means deploy is skipped. Not warned about -- skipped.

IN PLAIN WORDS

Every push runs six inspectors. Each one either stamps the build or slams the gate. The gate is wired to all six — so a failure on the left makes the release impossible on the right.

Why six gates and not one

each gate inspects something the others structurally cannot see

Why six gates and not one

each gate inspects something the others structurally cannot see

hadolint	sast	sca	gitleaks	image-scan
the recipe how the container is built	code you wrote bugs in lines you typed	code you borrowed known CVEs in libraries	values committed keys, tokens, passwords	the artifact OS + base image, invisible to SCA

Run 2: sca passed and image-scan failed on the same commit. Trivy found CVEs in wheel and jaraco.context -- packages that appear nowhere in requirements.txt.

Gate 1 — hadolint

lint the recipe before you turn on the oven

- **What it is:** a linter for your Dockerfile — the recipe that builds the container.
- **Named:** "Hado" from Docker's lineage + "lint", meaning to pick fluff off code.
- **Catches:** DL3002 (the container runs as root), DL3007 (an unpinned base image).
- **Why it earns its place:** it runs in parallel with the build, so a bad recipe fails in five seconds instead of after a two-minute build.

ANALOGY

Checking a cake recipe before turning the oven on. If it says "bake at 5000 degrees", you want to know that now — not after the kitchen is on fire.

Gate 2 — sast (Bandit)

flaws in the code you wrote

- **Stands for:** Static Application Security Testing. "Static" means the code is not running — it is read like an essay.
- **Catches:** dangerous patterns in your own source — `shell=True`, `eval`, unsafe deserialisation.
- **Found here:** B602 in run 1. A URL parameter flowed straight into a system shell, so `?cmd=...` was remote code execution.
- **The key limit:** a library scanner cannot find these. The flaw is in a line you typed.

ANALOGY

An English teacher marking your essay with a red pen — spotting forbidden constructions without ever reading the essay aloud.

Gate 3 — sca (pip-audit)

flaws in the code you borrowed

- **Stands for:** Software Composition Analysis — what your software is composed of.
- **Catches:** known CVEs in third-party packages, resolved from requirements.txt.
- **Found here:** 74 vulnerabilities across 8 packages, including a requests version that leaks your credentials on a redirect.
- **The point:** your own code can be flawless and this still fires. You do not control the supply chain.

ANALOGY

You did not grow the lettuce — you bought it. This inspector checks the barcode against the national recall list before it goes in the salad.

Gate 4 — gitleaks

the secrets you committed

- **Catches:** API keys, tokens and passwords sitting in files or commit history.
- **Why it is essential:** nothing else here reads values. SAST reads patterns, SCA reads version numbers, Trivy reads packages — an AWS key in a config file slides past all three.
- **Found here:** a planted test secret failed the gate and blocked the release, and the notification fired.
- **Redaction matters:** the scanner redacts the secret in its own output, so the CI log does not become the leak.

ANALOGY

Checking your backpack before you leave the house, to be sure you did not tape your house keys and door PIN to the front of your diary.

Gate 5 — image-scan (Trivy)

the finished artifact

- **Inspects:** the assembled container — OS packages, language packages, and everything the base image dragged in.
- **Why it is different:** it is the only gate that sees the object you actually ship, rather than the inputs to it.
- **Configured carefully:** only fixable findings fail it (ignore-unfixed), because 44 unfixable operating-system CVEs would make the gate permanently red.
- **Found here:** in run 2 it failed while sca passed — the CVEs were in the base image, invisible to the dependency scanner.

ANALOGY

Airport baggage X-ray. The officer does not read your packing list — the suitcase goes through the machine and they see what is physically inside.

The gate itself

one line of YAML that GitHub enforces for you

The gate is the dependency graph

one line of YAML, enforced by GitHub, not by discipline

deploy:

needs: [build, hadolint, sast, sca, gitleaks, image-scan]

GitHub will not start this job unless every
listed job succeeded. No script. No review.

any gate green

the door opens

any gate red

the door stays shut

Most teams have a policy: "we review for security before release."

This is a mechanism. Nobody has to remember anything.

Deploy is not a gate — it is the treasure

plus the alarm that fires when a gate shuts

- **deploy:** publishes the approved image to GHCR. It declares all six gates in needs:, so GitHub will not start it unless every one succeeded.
- **notify:** the inverse — if: failure(). Silent when everything is green, fires the moment something is blocked.
- **Why notify matters:** without it a red run just sits there until somebody happens to look. It writes a per-gate table and can post to Slack.
- **The separation:** the reusable workflow produces a scanned image; only the repository knows where its own image belongs.

THE SECURITY CONTROL

needs: is not a comment, a convention, or a review step. It is the dependency graph, and GitHub enforces it.

One artifact, promoted

the bytes that were scanned are the bytes that run

One artifact, promoted

the bytes that were scanned are the bytes that run



The obvious mistake: rebuilding the image inside the deploy job. That scans artifact A and ships artifact B -- the scan becomes theatre.

Verified in this project: CI pushed sha256:0a8d320d..., and the host runs sha256:0a8d320d...

The evidence

the gate is only real if you can show it shutting

Four runs, one story

the gate is only real if you can show it shutting

	build	hadolint	sast	sca	gitleaks	image-scan	deploy
Run 1 vulnerable baseline		not yet added			not yet added		
Run 2 deps patched		not yet added			not yet added		
Run 3 fully fixed							
Run 5 planted secret							

Run 2 is the important one: sca green, image-scan red, same commit. The gates are not redundant.

Run 2 is the slide that matters

sca green, image-scan red, same commit

- **What happened:** we patched requirements.txt. The dependency gate went green and we expected everything to follow.
- **What actually happened:** Trivy still failed — it found CVEs in wheel and jaraco.context, which ship inside the base image.
- **Why it could not be caught elsewhere:** those packages appear nowhere in requirements.txt. The dependency scanner is structurally blind to them.
- **What it proves:** the gates catch disjoint failure classes. One scanner would have shipped this.

THE SENTENCE TO REMEMBER

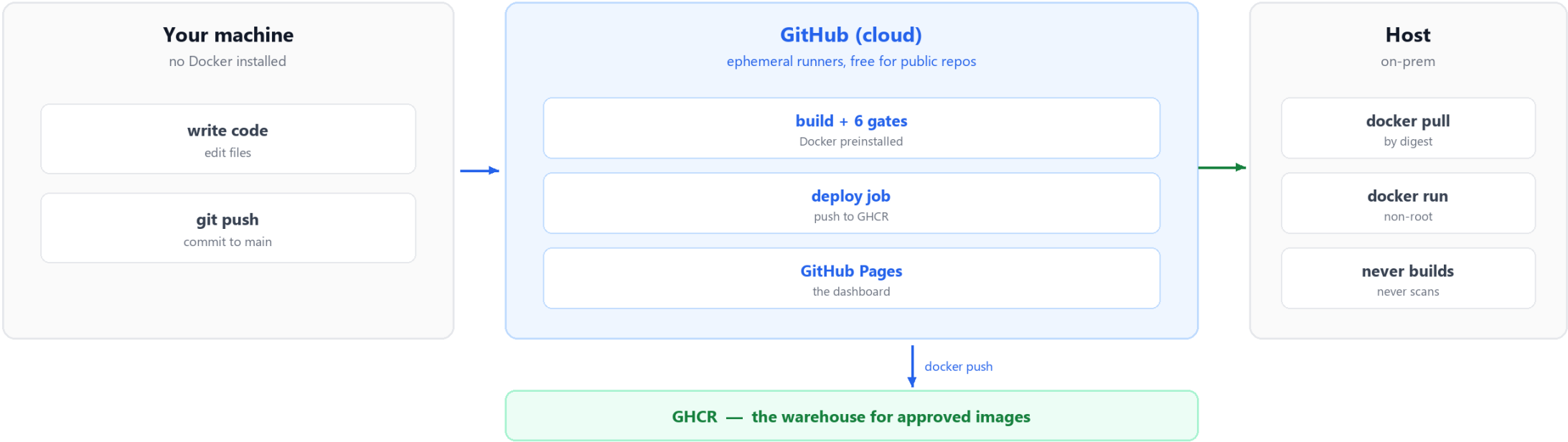
This is the difference between wiring up three scanners and understanding why three are needed.

What runs where

GitHub's runners do the work; the host only pulls and runs

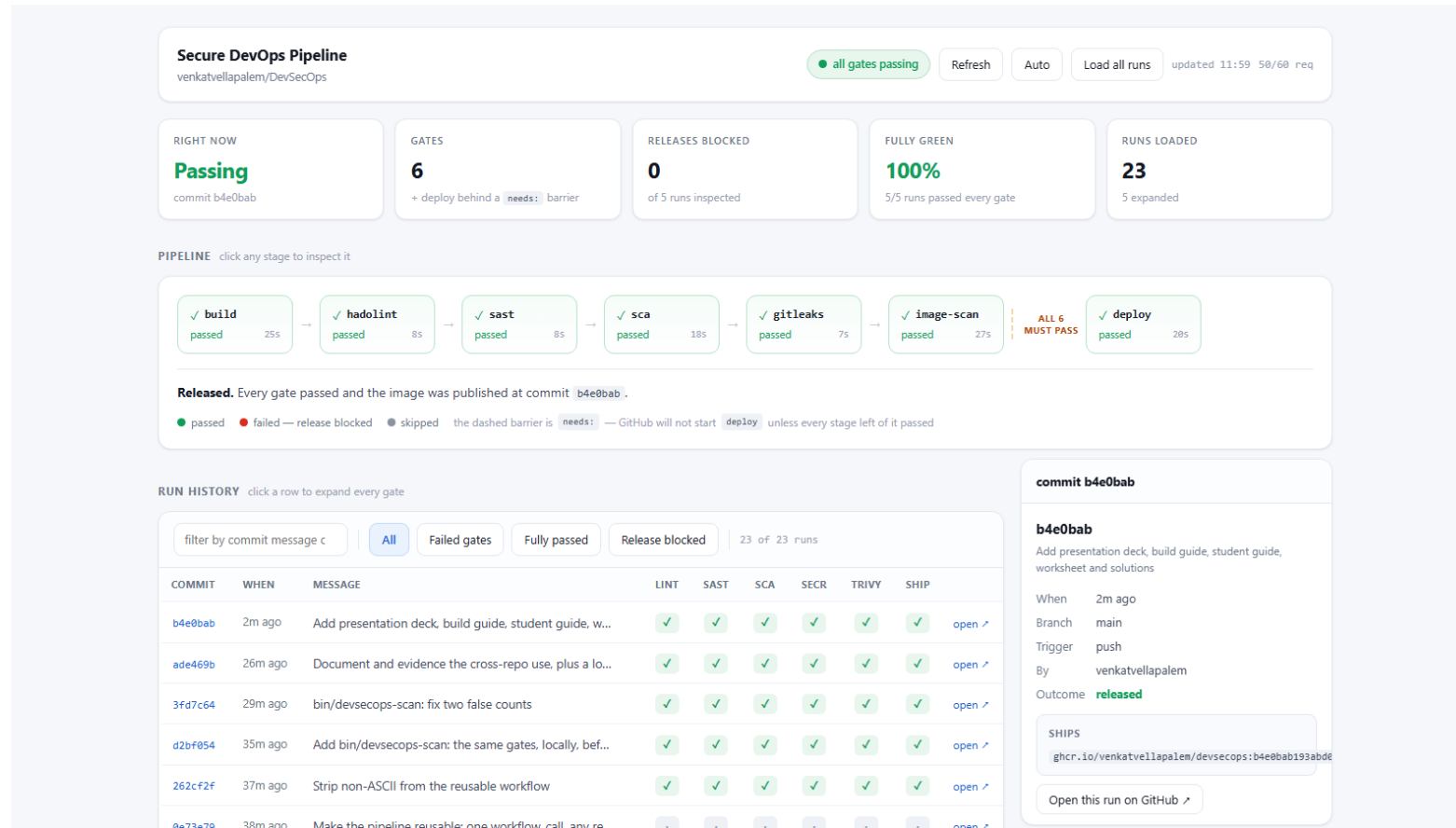
What runs where

GitHub's runners do all the work; the host only pulls and runs



A live dashboard, for humans

because a green pipeline nobody looks at is not reassurance



What you are looking at

One self-contained HTML file. No framework, no build step, no server to run.

Live

Reads the public GitHub API. The numbers are real, not a mockup.

Interactive

Click any stage for what it checks. Click any run to expand every job.

Why it exists

A manager should not need to read YAML to answer: are we safe to ship today?

<https://venkatvellingapalem.github.io/DevSecOps/>

We did not just follow the brief — we audited it

ten defects found by running it, not reading it

- **Two gates could never pass.** The spec's Trivy config failed on 44 unfixable operating-system CVEs, and its Bandit config tripped on its own sample code.
- **One rubric item was unwinnable.** The brief plants a vulnerable dependency for SCA but nothing for SAST, so "demonstrably functional SAST gate" could never be shown.
- **It scanned one artifact and shipped another.** The deploy job rebuilt the image, so the scan was theatre.
- **It ran an unpinned third-party action.** @master, with access to the workflow token, inside a project about supply-chain security.

WHY THIS MATTERS

Two gates that are always red teach people to bypass gates. A gate people route around is worse than no gate at all.

What this does not catch

say this before you are asked

- **Runtime attacks.** A clean image can still be exploited while running. This is a build-time check, not a firewall and not monitoring.
- **Logic flaws.** Broken access control and business-logic abuse look perfectly fine to a pattern matcher.
- **Configuration drift.** Nothing here enforces that production matches the repository.
- **Time.** The vulnerability database changes continuously. The same commit can pass today and fail next month with no code change — run 2 proved it.

WHY VOLUNTEER THIS

A green pipeline is a statement about a moment, not a guarantee. Being caught not knowing that reads far worse than saying it first.

Running it on our own hardware

pull-based, pinned by digest

- **Deploy by digest, not by tag.** latest is mutable — pulling it an hour later can hand you different bytes than the ones that were scanned.
- **Pull, do not push.** The host polls the registry. No inbound port, and no self-hosted runner.
- **Why not a self-hosted runner:** this repository is public, and a runner executes workflow code from any fork's pull request.
- **Rollback is a digest.** `./deploy/update.sh --rollback` returns to the previous image in seconds, without rebuilding anything.

HARDENING ALREADY APPLIED

non-root (uid 10001) · cap_drop: ALL · no-new-privileges · healthcheck · restart unless-stopped

Any repository can adopt this in 12 lines

no scanner installs, no configuration, no secrets

```
jobs:
  gates:
    uses: venkatvellapalem/DevSecOps/.github/workflows/security-gates.yml@v1
  with:
    dockerfile: Dockerfile · source-path: src
    requirements: requirements.txt
```

- **Verified.** Tested from a separate repository with an unrelated app — it found B602 in their code, 40 dependency CVEs, and fixable image CVEs.
- **Portable because** the github context comes from the caller, so checkout pulls their code, not ours.

THE TRAP

GITHUB_TOKEN permissions do not cross a workflow boundary — a caller wanting to push must grant packages: write itself.

The vocabulary, in one place

so nobody has to nod along at an acronym

CI / CD — automatically build and ship on every push

SAST — scan the source you wrote

SCA — scan the dependencies you consumed

artifact — the built thing — here, a container image

digest — a fingerprint of exact bytes; two images sharing one are identical

registry — the warehouse images live in — GHCR

immutable — built once, then never modified, only promoted

provenance — proof an artifact is the one that was checked

shift left — move a check earlier, where it is cheap

fail fast — stop at the first problem instead of carrying it forward

least privilege — grant only the access actually needed

A gate people route around is worse than no gate at all.

Six gates that can each be shown failing.

Before-and-after evidence from real runs.

Provable provenance from commit to running container.

Questions?